Ukrainian Catholic University

Faculty of Applied Sciences

Semantic Document Retrieval

Authors:
Kalmuk Yaropolk
Mykolaichuk Nazar
Talan Theodor

Mentor:
Lyaskovska Nadia

1 Research Area and Importance

The research area of this project lies at the intersection of **Information Retrieval (IR)** and **Natural Language Processing (NLP)**. In an era defined by the exponential growth of digital data, the ability to efficiently navigate and extract knowledge from massive, unstructured document collections has become an important part of modern data science.

At its core, a **Document Retrieval System** is a technical framework designed to find and rank the most relevant pieces of information from a large collection (a corpus) based on a user's query. Today, many retrieval systems may rely on the Simple Vector Space Model, in which documents are represented as vectors in a high-dimensional space based just on word frequencies. However, these traditional keyword-based models suffer from a fundamental flaw: they operate on surface-level character matching rather than conceptual meaning. This leads to a problem, where a system fails to retrieve a relevant document simply because the author and the user chose different synonyms for the same idea.

This research is significant because it implements a **Latent (Hidden) Semantic framework** to bridge this gap. By moving beyond literal word-counting toward a deep semantic understanding, our system "reads between the lines" to identify contextual relationships. This allows for a more intuitive search experience where results are determined by the underlying theme of the text rather than a simple overlap of specific characters.

2 Project Aim and Tasks

The primary aim of this project is to develop an automated semantic document retrieval system that identifies the most suitable documents for a user's query. Unlike standard search engines, our system uses techniques to map the meaning of a text rather than just its spelling. To achieve this, we have defined the following technical tasks:

- **Data Pre-processing:** Tokenize raw text and apply TF-IDF weighting to represent documents as high-dimensional vectors.
- **Latent Semantic Analysis:** Implement SVD to decompose the term-document matrix and identify hidden conceptual relationships.
- **Dimensionality Reduction:** Truncate the decomposed matrices to a k-rank approximation to filter noise and map documents into a "Concept Space."
- **Query Matching:** Develop a retrieval mechanism that projects user queries into this semantic space and identifies the most suitable documents using cosine similarity.

3 Possible Approaches and Available Solutions

In the field of Information Retrieval, the most basic approach of how to look at text is the Boolean Retrieval model, which only considers the presence or absence of a word. A more advanced solution is the standard Vector Space Model using pure TF-IDF. It's purpose is to give every word its own "weight" - an importance in a specific document. Words with higher weights are more significant than the ones with lower weights in a

chosen document. TF-IDF effectively gives more weight to words that appear frequently within a specific text but rarely across the entire collection. But with this approach only, if we set a document as a vector of those weights, it creates orthogonal axes for every unique word. This means synonyms have a dot product of zero, indicating no similarity. It treats every word as an independent, orthogonal axis. Consequently, synonyms like "automobile" and "car" have a dot product of zero, indicating no similarity despite their shared meaning.

So, to solve this, our project implements **Latent Semantic Analysis (LSA)** powered by **Singular Value Decomposition (SVD)**. This method decomposes the term-document matrix into three components ($A \approx U\Sigma V^T$), allowing us to analyze the correlation between documents ($A^T A$) and terms (AA^T). By truncating the diagonal matrix Σ to its top k singular values, we reduce the dimensionality of the data, forcing semantically related terms to project onto the same axes.

We chose LSA because it provides a mathematically rigorous way to uncover the hidden structure of language using linear algebra. Unlike deep learning models, LSA is highly efficient for smaller datasets and offers clear geometric interpretability.

Recommended Literature and Resources

To support our implementation, we rely on the following foundational works and technical tutorials:

- **Video Resource:** *Term Frequency Inverse Document Frequency (TF-IDF) Explained* by DataMListic. This is how we construct our matrix with TF-IDF method.
- **Literature:** *Indexing by Latent Semantic Analysis* by Deerwester et al. (1990). This is the foundational paper that introduced the use of SVD for document retrieval.
- **Literature:** *Introduction to Information Retrieval* by Manning et al. Chapter 18 specifically covers Matrix Decompositions and LSA.
- **Video Resource:** *Singular Value Decomposition (SVD) - Overview* by Steve Brunton. An introduction to SVD.
- **Video Resource:** *Singular Value Decomposition (SVD): Mathematical Overview* by Steve Brunton. A excellent mathematical overview of SVD.
- **Video Resource:** *Linear Algebra - SVD Applications* by MIT OpenCourseWare. Provides the mathematical rigor behind dimensionality reduction.

4 Pseudocode of the Implementation

The system's logic is divided into distinct mathematical phases:

Phase 1: Feature Extraction and Weighting.
In the context of this project, a **corpus** is defined as a collection of authentic texts. Rather than being a simple set of disconnected sentences, the corpus represents a structured database of **documents** (e.g., articles, reports, or chapters).

The first step in our pipeline is **Preprocessing**, which involves tokenizing the text and performing **Stop-word Removal**. By filtering out "filler" words (high-frequency terms with low semantic value, such as "is," "the," and "at"), we ensure that our mathematical model focuses only on the meaningful terms that define a document's topic.

We represent the processed corpus as a Term-Document Matrix $A \in \mathbb{R}^{m \times n}$, where m is the number of unique terms (words), excluding filtered stop-words, and n is the total number of documents. We then calculate the **TF-IDF** (Term Frequency-Inverse Document Frequency) weight for each entry $A_{i,j}$ as follows:

$$A_{i,j} = TF_{i,j} \times IDF_i \tag{1}$$

These values are placed in matrix A, forming the high-dimensional vector space from which the semantic concepts will be extracted.

- **Term Frequency** $TF(i,j)$: This measures the local importance of term i within document d_j. It is normalized by the total number of terms in the document to prevent a bias toward longer texts:

$$TF(i,j) = \frac{f_{i,j}}{\sum_k f_{k,j}} \tag{2}$$

  where $f_{i,j}$ is the raw count of term i in document d_j, and the denominator represents the total word count of that document.

- **Inverse Document Frequency (IDF_i):** This measures the global significance of a term across the entire corpus D. It is calculated to penalize common words that appear in almost every document, and so, doesn't actually mean anything to us:

$$IDF(i) = \log\left(\frac{|D|}{|\{j \in D : \text{term } i \in d_j\}|}\right) \tag{3}$$

  where $|D|$ is the total number of documents in the collection, and the denominator represents the document frequency (df_i), or the number of documents containing at least one occurrence of term i. The logarithmic scaling ensures that the weight does not grow too rapidly as the corpus size increases.

Phase 2: Low-Rank Approximation (SVD).
The core of Latent Semantic Analysis (LSA) is the decomposition of the $m \times n$ term-document matrix A into three constituent matrices: $A = U\Sigma V^T$. This factorization identifies the underlying structure of the language by analyzing the correlations between terms and documents.

Each matrix in the decomposition represents a specific dimension of the semantic space:

- **Right Singular Vectors ($V \in \mathbb{R}^{n \times n}$):** The Right Singular Vectors form the columns of the matrix V. These vectors represent the **Document-Concept** space, defining how individual documents align with latent semantic themes. To find these vectors, we utilize the relationship between the SVD of matrix A and the Eigenvalue Decomposition of the symmetric matrix $A^T A$.

We begin by multiplying the Term-Document matrix transpose A^T by A. Since A is an $m \times n$ matrix, this results in a symmetric $n \times n$ matrix that represents the correlations between documents across the entire corpus:

$$S = A^T A \tag{4}$$

Using the definition of SVD, $A = U\Sigma V^T$, we substitute this into the correlation matrix:

$$A^T A = (U\Sigma V^T)^T (U\Sigma V^T) \tag{5}$$

Applying the transpose rule $(ABC)^T = C^T B^T A^T$:

$$A^T A = (V\Sigma^T U^T)(U\Sigma V^T) \tag{6}$$

Since U is an orthogonal matrix, $U^T U = I$ (the identity matrix). Thus, the equation simplifies to:

$$A^T A = V\Sigma^T \Sigma V^T = V\Sigma^2 V^T \tag{7}$$

The resulting equation $A^T A = V\Sigma^2 V^T$ is the standard form of an Eigenvalue Decomposition. This implies that the columns of V are the eigenvectors of $A^T A$. To find them, in theory we solve the characteristic equation:

$$\det(A^T A - \lambda I) = 0 \tag{8}$$

where λ represents the eigenvalues of the document correlation matrix.

For each eigenvalue λ_i, we solve the following system of linear equations to find the corresponding eigenvector v_i:

$$(A^T A - \lambda_i I)v_i = 0 \tag{9}$$

The set of orthonormal eigenvectors $\{v_1, v_2, \ldots, v_n\}$ constitutes the columns of our complete matrix $V \in \mathbb{R}^{n \times n}$.

Numerical Implementation: Power Iteration and Deflation

To compute these eigenvectors efficiently in our code without relying on external libraries, we implemented the **Power Iteration** algorithm combined with **Hotelling's Deflation**.

1. **Power Iteration:** To find the dominant eigenvector of the correlation matrix $S = A^T A$, we implement an iterative process. For each latent concept, the following steps are performed:
   (a) **Initialization:** Generate a random starting vector $b_0 \in \mathbb{R}^n$. This vector acts as an initial guess for the eigenvector.
   (b) **Matrix-Vector Multiplication:** Compute the product $w = Sb_k$. This operation rotates the vector b_k toward the direction of the eigenvector associated with the largest eigenvalue.
   (c) **Normalization:** To prevent the vector's magnitude from growing to infinity (overflow) or shrinking to zero (underflow), we normalize it to unit length:

$$b_{k+1} = \frac{w}{\|w\|} \tag{10}$$

(d) **Convergence Check:** We repeat steps (b) and (c) until the change between iterations becomes negligible:

$$\|b_{k+1} - b_k\| < \epsilon \tag{11}$$

where ϵ is a predefined tolerance (e.g., 10^{-9}).

(e) **Eigenvalue Extraction:** Once converged, the corresponding eigenvalue λ is calculated using the Rayleigh Quotient:

$$\lambda = \frac{b^T S b}{b^T b} = b^T S b \quad (\text{since } \|b\| = 1) \tag{12}$$

This process identifies the most significant "latent concept" in the corpus by finding the direction of maximum variance in the document-term space.

2. **Hotelling's Deflation:** Once the dominant eigenvector v_1 and its eigenvalue λ_1 are found, we remove its influence from the matrix to find the subsequent concept. We update the matrix S as follows:

$$S_{next} = S - \lambda_1 v_1 v_1^T \tag{13}$$

By repeating this process, we can successively extract the orthogonal eigenvectors that define our semantic space.

- **Left Singular Vectors ($U \in \mathbb{R}^{m \times m}$):** The Left Singular Vectors represent the **Term-Concept** space. Rather than performing another computationally expensive Eigenvalue Decomposition on the massive $m \times m$ correlation matrix AA^T, our implementation derives the matrix U algebraically from the fundamental definition of SVD.

  Starting with the base SVD equation:

$$A = U\Sigma V^T \tag{14}$$

We right-multiply both sides by the orthogonal matrix V:

$$AV = U\Sigma V^T V \tag{15}$$

Because V is an orthogonal matrix, its multiplication with its transpose yields the identity matrix ($V^T V = I$). This simplifies the relationship to:

$$AV = U\Sigma \tag{16}$$

This matrix equation can be evaluated column by column. The i-th column of AV is given by AV_i. On the right side, since Σ is a diagonal matrix, it simply scales the i-th column of U by the corresponding singular value Σ_i. Therefore, for each corresponding column:

$$AV_i = U_i \Sigma_i \tag{17}$$

Finally, to isolate the left singular vector U_i, we divide by the scalar singular value Σ_i (for all non-zero singular values):

$$U_i = \frac{AV_i}{\Sigma_i} \tag{18}$$

Geometrically, this means we map the document-space concept vector V_i into the term space via A, and then normalize its length to 1 by dividing by Σ_i, yielding the orthonormal columns of the full matrix $U \in \mathbb{R}^{m \times m}$.

- **Singular Values ($\Sigma \in \mathbb{R}^{m \times n}$):** The Singular Values are the diagonal elements of the rectangular matrix Σ, representing the **magnitude** or strength of each latent concept identified by the SVD. To find these values, we utilize the eigenvalues (λ) derived from the characteristic equations of either AA^T or A^TA.

  As demonstrated in the derivations for U and V, both the term-term and document-document correlation matrices share the same non-zero eigenvalues λ_i. The relationship between these eigenvalues and the singular values σ_i is defined as:

  $$\sigma_i = \sqrt{\lambda_i} \tag{19}$$

  Once the square roots are calculated, the singular values are placed along the main diagonal of the Σ matrix in descending order:

  $$\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0 \tag{20}$$

  where $r \leq \min(m, n)$ is the rank of the matrix A.

Dimensionality Reduction and Truncation

In practical Natural Language Processing applications, computing the full SVD is computationally prohibitive and semantically undesirable. Because a massive portion of the raw term-document matrix A consists of linguistic noise (e.g., typographical errors, stylistic variations, and rare words), keeping all dimensions leads to overfitting.

To resolve this, we truncate our SVD to rank $k \ll \min(m, n)$. Instead of letting our Power Iteration and Deflation algorithm run for all n document dimensions, we actively halt the loop after precisely k iterations. This allows us to construct a low-rank approximation of the original term-document matrix:

$$A \approx A_k = U_k \Sigma_k V_k^T \tag{21}$$

This truncation yields a highly compressed semantic space with the following properties:

1. **Matrix Dimensions:** The massive, sparse matrices are projected into dense, lower-dimensional representations:
   - **Truncated Left Singular Vectors ($U_k \in \mathbb{R}^{m \times k}$):** Contains only the top k columns of U, representing the term-concept mapping.
   - **Truncated Singular Values ($\Sigma_k \in \mathbb{R}^{k \times k}$):** A small, square diagonal matrix containing the k largest singular values in descending order.
   - **Truncated Right Singular Vectors ($V_k^T \in \mathbb{R}^{k \times n}$):** Contains only the top k rows of V^T, representing the document-concept mapping.
2. **Computational Savings during SVD Execution:** By targeting a specific k, we only run the Power Iteration and Hotelling's Deflation loop k times. This means we only need to calculate k eigenvectors rather than the full n eigenvectors. Mathematically, this reduces our execution time complexity from $O(n^3)$ for a complete decomposition to $O(k \cdot n^2)$ for a partial SVD, significantly reducing CPU cycles and thermal overhead.

Phase 3: Semantic Projection into Concept Space Once the SVD decomposition and truncation are complete, we must project the original documents and incoming queries into the newly discovered k-dimensional semantic space.

The relationship $A \approx U_k \Sigma_k V_k^T$ implies that the document coordinates in the reduced space are represented by:

$$D_{sem} = \Sigma_k V_k^T \tag{22}$$

In this matrix $D_{sem} \in \mathbb{R}^{k \times n}$, each column represents a document d_j scaled by the importance of the k latent concepts.

The mathematical justification for this projection lies in the change of basis from the high-dimensional term space to the low-dimensional concept space. Since the columns of U_k provide an orthonormal basis for the semantic space, we project the original document vectors in A onto this basis by left-multiplying by U_k^T:

$$D_{sem} = U_k^T A \tag{23}$$

By substituting the SVD fundamental identity ($A \approx U_k \Sigma_k V_k^T$) into this projection equation, we can derive the reduced representation:

$$D_{sem} = U_k^T (U_k \Sigma_k V_k^T) \tag{24}$$

Using the associative property of matrix multiplication:

$$D_{sem} = (U_k^T U_k) \Sigma_k V_k^T \tag{25}$$

A critical property of the SVD is that U is an orthogonal matrix, meaning its columns are orthonormal. Therefore, $U_k^T U_k = I$, where I is the identity matrix. This simplifies the expression to:

$$D_{sem} = I \Sigma_k V_k^T = \Sigma_k V_k^T \tag{26}$$

This proof demonstrates that the coordinates of our documents in the semantic space are represented simply by the product of the singular values and the right singular vectors.

Phase 4: Query Projection and Semantic Matching.
A new user query q is initially a sparse vector in the original term space. To compare it against our semantic map, we must project it into the k-dimensional concept space to create q_{sem}.

- **Query Projection:**
  To search the system, a user's query q is first represented as a vector of TF-IDF weights for the query terms. Since this query was not part of the original SVD calculation, we must project it into the k-dimensional concept space using the term-concept matrix U_k:

  $$q_{sem} = U_k^T q \tag{27}$$

  This operation transforms the query by mapping its constituent terms onto the latent axes discovered during the decomposition. This allow the system to retrieve relevant results even if the query and documents share no identical keywords.

- **Cosine Similarity:**
  Once the query and documents exist in the same k-dimensional space, we calculate the similarity between q_{sem} and each document vector d_j (where d_j is a column of D_{sem}):

$$\text{similarity} = \cos(\theta) = \frac{q_{sem} \cdot d_j}{\|q_{sem}\|\|d_j\|} \tag{28}$$

  This measures the cosine of the angle between the two vectors, focusing purely on their direction. If two vectors point in the same direction in the concept space, they share the same thematic meaning, regardless of their original length or specific word overlap.

Phase 5: Ranking and Result Retrieval:
The final step is to generate a sorted list of documents based on their relevance. For a given query q_{sem}, we compute a set of similarity scores $S = \{s_1, s_2, \ldots, s_n\}$, where each $s_j = \text{sim}(q_{sem}, \mathbf{d}_j)$.

The results are retrieved by identifying the indices of the documents that maximize the similarity score:

$$\text{Result} = \text{sort\_descending}(\{(d_j, s_j) \mid j = 1, \ldots, n\}) \tag{29}$$

By selecting the top N documents with the highest cosine values, the system surfaces the texts that are geometrically closest to the query.

5 Testing and Data

To evaluate the effectiveness of our LSA implementation, we utilized the BBC News dataset, a respected benchmark consisting of 2,225 documents from the BBC news website of 2004-2005 year. The corpus is categorized into five distinct thematic areas: Business, Entertainment, Politics, Sport, and Tech. This dataset provides an ideal environment for testing how well Singular Value Decomposition (SVD) identifies latent thematic clusters, such as distinguishing between specific types of athletes or political conflicts.

To evaluate the performance of the LSA retrieval system, we conducted qualitative testing through manual query analysis. This involved selecting a set of representative topics and comparing the retrieved documents against the user's conceptual intent rather than literal keyword overlap.

We focused on two specific types of queries to validate the system's mathematical depth:

- **Direct Keyword Queries:** We input terms explicitly present in the corpus to ensure that the TF-IDF weighting and Cosine Similarity correctly identified high-frequency matches.

- **Latent Semantic Queries (Synonymy/Polysemy):** We input related concepts to verify that the SVD successfully mapped the query to the correct "Concept Space." For example, querying for *"football forward"* successfully retrieved documents concerning specific strikers like *Thierry Henry* and *Ruud van Nistelrooy*, as well as articles using the term *"striker,"* despite the word *"forward"* not being the primary descriptor in those texts.

A test is considered successful if the top-ranked results exhibit a high cosine similarity score (typically > 0.70) and contain thematic content that matches the query's subject matter. Our results confirmed that by truncating the matrix to $k = 50$, the system successfully filtered out linguistic noise, allowing for effective retrieval based on underlying meaning rather than simple lexical overlap.

6 Testing and Data

To evaluate the effectiveness of our LSA implementation, we utilized the BBC News dataset, a respected benchmark consisting of 2,225 documents from the BBC news website of the 2004-2005 year. The corpus is categorized into five distinct thematic areas: Business, Entertainment, Politics, Sport, and Tech. This dataset provides an ideal environment for testing how well Singular Value Decomposition (SVD) identifies latent thematic clusters, such as distinguishing between specific types of athletes or political conflicts.

To evaluate the performance of the LSA retrieval system, we conducted qualitative testing through manual query analysis. This involved selecting a set of representative topics and comparing the retrieved documents against the user's conceptual intent rather than literal keyword overlap.

We focused on two specific types of queries to validate the system's mathematical depth:

- **Direct Keyword Queries:** We input terms explicitly present in the corpus to ensure that the TF-IDF weighting and Cosine Similarity correctly identified high-frequency matches.
- **Latent Semantic Queries (Synonymy/Polysemy):** We input related concepts to verify that the SVD successfully mapped the query to the correct "Concept Space." For example, querying for *"football forward"* successfully retrieved documents concerning specific strikers like *Thierry Henry* and *Ruud van Nistelrooy*, as well as articles using the term *"striker,"* despite the word *"forward"* not being the primary descriptor in those texts.

Hyperparameter Selection: Determining the Optimal k

A critical phase of our system evaluation was determining the optimal number of latent concepts, k. Selecting a k that is too low results underfitting, where distinct semantic topics are compressed into the same coordinate axes. Conversely, selecting a k that is too high retains stylistic noise, rare vocabulary, and typographical errors, causing the model to behave like a standard, literal keyword search.

To systematically identify this threshold, we analyzed the decay of the singular values σ_i generated during SVD. Mathematically, the proportion of total variance (or information) retained by a rank-k approximation is defined by the cumulative energy ratio:

$$\text{Variance Reclaimed}(k) = \frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2} \tag{30}$$

where r is the full rank of the term-document matrix A.

By plotting these singular values in descending order, we observed a steep initial drop-off representing the dominant thematic categories of the BBC dataset, followed by a gradual flattening.

Our analysis showed that at $k = 50$, the system captures approximately **64.8% of the total variance** of the entire dataset. This configuration preserves the core semantic relationships of the corpus while successfully discarding the remaining 35.2% of the variance, which is dominated by random word-co-occurrence noise. This mathematical inflection point can be clearly observed in our system diagnostic plots, validating our choice of $k = 50$ as the optimal dimensional bottleneck.

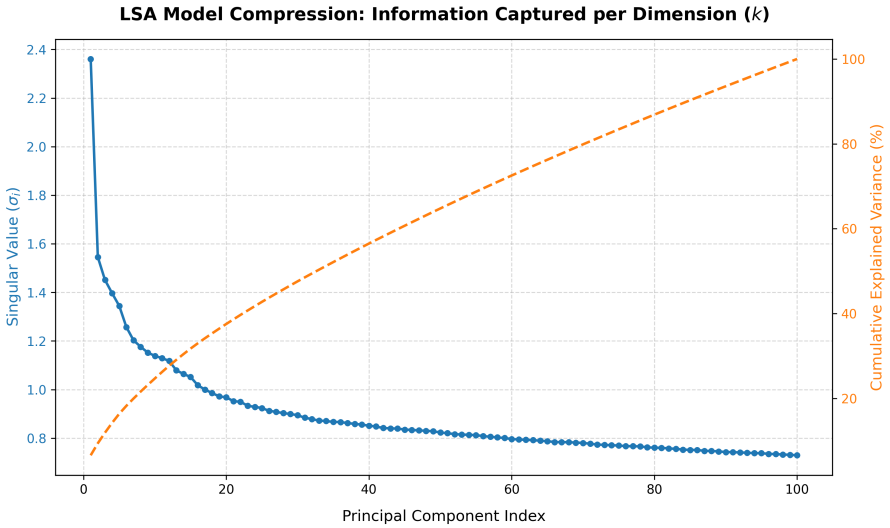

Figure 1: Scree plot of singular values and cumulative variance explained for the BBC News dataset, indicating the chosen $k = 50$ elbow threshold.

Retrieval Evaluation

A test is considered successful if the top-ranked results exhibit a high cosine similarity score (typically > 0.70) and contain thematic content that matches the query's subject matter. Our results confirmed that by truncating the matrix to $k = 50$, the system successfully filtered out linguistic noise, allowing for effective retrieval based on underlying meaning rather than simple lexical overlap.

7 Challenges and Limitations

While our implementation successfully demonstrates the power of Latent Semantic Analysis on the BBC corpus, scaling the system and applying it to broader linguistic tasks introduces several distinct challenges:

- **Hyperparameter Sensitivity of k:** Determining the optimal number of dimensions k remains a critical, empirical challenge. As demonstrated in our testing, this hyperparameter acts as a harsh trade-off filter. If k is too high, the system retains stylistic and vocabulary noise, causing it to degrade back to literal, rigid keyword matching. If k is too low, critical semantic distinctions are lost, forcing unrelated topics (such as different field sports) to merge into the same concept axes.

- **Loss of Syntactic Structure (Bag-of-Words Limitation):** Because Phase 1 of our pipeline constructs a Term-Document matrix based purely on raw term frequencies, LSA completely discards word order, syntax, and grammar. The system cannot distinguish between sentences with identical words but opposite meanings (e.g., *"the manager blamed the player"* versus *"the player blamed the manager"*). This lack of syntactic awareness limits the engine's precision on structurally complex queries.

- **Sensitivity to Negation:** A consequence of the bag-of-words representation is the system's inability to handle negation. Because cosine similarity focuses on vocabulary co-occurrence in the concept space, a document containing *"this article is about football"* and one stating *"this article is not about football"* will project to nearly identical coordinates in the latent space.

- **Data Sparsity and Document Length:** The mathematical integrity of SVD relies heavily on the "Co-occurrence Principle"—the assumption that semantically related terms regularly appear together within the same document boundaries. If the target dataset consists of exceptionally short documents (such as social media micro-posts) or documents that share virtually zero overlapping vocabulary, the term-document matrix A becomes too sparse. In such cases, the SVD fails to capture meaningful latent dimensions, causing a collapse in retrieval performance.

8 Implementation Architecture

Github Repository

The system is implemented as a modular pipeline in Python, utilizing `NumPy` for high-performance matrix operations and `Pandas` for dataset management.

8.1 Module Breakdown

- **Data Processing** (`data_prep.py`): This module handles the transformation of the BBC News corpus into a Term-Document Matrix. It performs tokenization, stop-word removal, and computes the TF-IDF weights to normalize document lengths.

- **SVD Engine** (`svd_model.py`): Contains our manual implementation of the SVD. It executes the Power Iteration and Hotelling's Deflation algorithms to derive the matrices U, Σ, and V^T without the use of external libraries.

- **Retrieval Logic** (`search.py`): Manages the query folding-in process and calculates Cosine Similarity scores between the query vector and the semantic document space.

8.2 Execution and User Interface (`main.py`)

The system is orchestrated through the `main.py` script. To interact with the engine, users modify the `if __name__ == "__main__":` block. This design allows for precise control over the following hyperparameters:

1. **Query Input**: The `user_query` string is defined here (e.g., *"football forward"*).

2. **Dimension Selection (k)**: Users can set the value of k for themselves to determine the depth of semantic compression.

3. **Top-N Results**: The variable `top_n` determines how many relevant documents the system returns to the console.

8.3 Results and Output (`results.txt`)

To ensure reproducibility and facilitate qualitative analysis, all search outputs are logged to `results.txt`. This file records the processed query, the k value used, and a list of the top n retrieved documents along with their corresponding Cosine Similarity scores. This allows for a rigorous comparison between the system's mathematical rankings and the human-perceived semantic relevance.

9 Implementation Architecture

Github Repository

The system is implemented as a modular pipeline in Python, utilizing `NumPy` for high-performance matrix operations and `Pandas` for dataset management.

9.1 Module Breakdown

- **Data Processing (`data_prep.py`)**: This module handles the transformation of the corpus into a Term-Document Matrix. It performs tokenization, stop-word removal, and computes the TF-IDF weights to normalize document lengths and establish the vocabulary.

- **SVD Engine (`svd_model.py`)**: Contains our custom, scratch-built implementation of SVD. Instead of performing a complete, memory-heavy decomposition and subsequently discarding dimensions, this engine utilizes Power Iteration with Hotelling's Deflation to directly compute only the top-k components. It outputs the low-rank truncated matrices U_k, Σ_k, and V_k^T in a single, highly optimized workflow.

- **Retrieval Logic (`search.py`)**: Manages semantic querying. It projects raw queries into the concept space via $q_{sem} = U_k^T q$. It then evaluates the semantic similarity between this projected query and the concepts-scaled document representation ($D_{sem} = \Sigma_k V_k^T$) using Cosine Similarity metrics.

9.2 Execution and User Interface (`main.py`)

The system is orchestrated through the `main.py` script. To interact with the engine, users modify the `if __name__ == "__main__":` block. This design allows for precise control over the following hyperparameters:

1. **Query Input**: The `user_query` string is defined here (e.g., *"football coach"*).

2. **Dimension Selection (k)**: Users can set the value of k to determine the depth of semantic compression.

3. **Top-N Results**: The variable `top_n` determines how many relevant documents the system returns to the console.

9.3 Results and Output (`results.txt`)

To ensure reproducibility and facilitate qualitative analysis, all search outputs are logged to `results.txt`. This file records the processed query, the k value used, and a list of the top n retrieved documents along with their corresponding Cosine Similarity scores. This allows for a rigorous comparison between the system's mathematical rankings and the human-perceived semantic relevance.